

Mycellius — Séance 3 (4h) : Sprint 2

Exceptions, collections, recherche simple, JUnit sur le modèle (ex: titre obligatoire, révision n+1)

Objectif global de la séance 3

À partir de ce que tu as déjà dans la séance 2 (Tag, WikiPage, tests JUnit simples, CI Maven/GitHub) on va :

- renforcer la robustesse du code (gestion d'erreurs, exceptions)
- introduire les collections (List, Map) pour stocker plusieurs pages
- créer un « mini service métier » pour Mycellius (création + recherche de pages)
- formaliser un début de contrat d'API : quelles méthodes existent, que renvoient-elles, que se passe-t-il en cas d'erreur...
- couvrir ce service par des tests JUnit (cas nominal + cas d'erreur)
- le tout en restant dans du Java « simple », mais prêt à être migré vers Spring Boot en séance 4

Pré-requis supposés (fin de séance 2)

Dans le module Maven api, tu as déjà :

- Tag (fr.mycellius.domain.Tag)
- WikiPage (fr.mycellius.domain.WikiPage)
- Tests JUnit TagTest et WikiPageTest dans src/test/java/fr/mycellius/domain
- Une classe App avec un main qui affiche "Mycellius démarre !" et fait une mini démo Tag/WikiPage
- Une CI GitHub Actions qui lance mvn -f api/pom.xml test sur les branches dev / test / prod

On ne repart donc pas de zéro : on va enrichir le même module api.

Plan détaillé de la séance 3 (4h)

- Rappel rapide séance 2 + positionnement séance 3 ($\approx 10\text{--}15$ min)
- Exceptions et robustesse : bien gérer les erreurs ($\approx 45\text{--}60$ min)
- Collections et dépôt en mémoire : InMemoryWiki ($\approx 45\text{--}60$ min)
- Mini service métier : WikiService (create + search) et contrat de méthodes ($\approx 45\text{--}60$ min)
- Tests JUnit sur le service (cas nominal + erreurs) ($\approx 45\text{--}50$ min)
- CI + workflow Git (dev $\rightarrow$ test) avec ce nouveau code ($\approx 20\text{--}30$ min)
- Budget « refactor qualité » ($\approx 15\text{--}20$ min, à faire tout au long)

1. Rappel séance 2 et objectif séance 3

1.1 Ce que tu as déjà

- Un projet Maven api propre, avec JDK 21 configuré
- Des classes métier simples (Tag, WikiPage) qui vérifient déjà quelques règles (titre obligatoire, tag non vide, etc.)
- Des tests JUnit qui sécurisent ces règles
- Une CI qui exécute mvn test sur GitHub

Idée importante :

En séance 2, tu as posé la « grammaire » de Java (types, variables, POO de base) et tu as commencé à modéliser le métier Mycellius.

En séance 3, on renforce la qualité :

- on s'occupe des erreurs (exceptions)
- on commence à manipuler des collections d'objets
- on crée un service métier qui ressemble déjà à ce que sera ton API plus tard.

1.2 Objectif concret de fin de séance

À la fin de cette séance, tu dois avoir :

- une classe InMemoryWiki qui stocke des WikiPage dans une collection
- une classe WikiService qui expose des méthodes comme :
 - createPage(id, title, content)
 - getPageById(id)
 - searchByTitle(fragment)
- des exceptions métiers claires (par exemple PageNotFoundException)
- des tests JUnit sur WikiService (cas qui marchent + cas d'erreur)
- une CI toujours verte sur GitHub

Ce WikiService sera le « contrat métier » que ta future API Spring exposera via HTTP.

2. Exceptions et robustesse (≈ 45–60 min)

2.1 Rappel : tu utilises déjà des exceptions

Dans Tag et WikiPage, tu lances déjà des IllegalArgumentException dans plusieurs cas :

- Tag.normalize : si raw est null ou vide après trim
- WikiPage : si id est null/vide, si title est null/vide

=> Ce sont des exceptions non vérifiées (RuntimeException).

=> Elles permettent de refuser très tôt des données incohérentes.

Idée importante :

=> Pour un service métier, il ne suffit pas de « lancer des exceptions au hasard ».

Il faut :

- documenter quand elles peuvent se produire (contrat de méthode)
- choisir des types d'exception qui ont du sens (IllegalArgumentException, ou une exception métier dédiée)
- décider s'il vaut mieux renvoyer un résultat optionnel (Optional, liste vide) ou lever une exception.

2.2 Créer une exception métier simple : PageNotFoundException

Objectif

- On veut une exception claire quand on cherche une page qui n'existe pas.
- Plus tard, en Spring Boot, cette exception sera mappée (associée/redirigée) vers un HTTP 404.

Dans IntelliJ, dans src/main/java/fr/mycellius/domain:

> crée un sous-package fr.mycellius.domain.exception (pour travailler proprement)

> puis crée un nouveau fichier : PageNotFoundException.java

```
package fr.mycellius.domain;

public class PageNotFoundException extends RuntimeException {
    public PageNotFoundException(String id) {
        super("Page introuvable pour l'id: " + id);
    }
}
```

Explication rapide

- La classe hérite de RuntimeException : on n'est pas obligé de la déclarer avec throws.
- Le constructeur prend un id et construit un message explicite.
- Plus tard, dans des logs ou une réponse d'API, ce message aidera à comprendre ce qui s'est passé.

=> On a maintenant une brique réutilisable pour tout ce qui cherche des pages.

2.3 Utiliser try / catch dans App pour illustrer le comportement

Objectif

- Montrer comment se comporte le code quand une PageNotFoundException est levée, et comment on peut l'intercepter.
- Dans App.main, ajoute un bout de code illustratif (tu ajusteras plus tard quand le service sera créé, mais pour l'instant on peut simuler) :

Quand j'écris "App.main", je parle simplement de la méthode suivante dans ton fichier App.java :

```
public static void main(String[] args) { ... }
```

Donc :

– App = le nom de la classe

– main = le nom de la méthode

=> App.main = "la méthode main de la classe App".

Exemple temporaire :

```
System.out.println("--- Démo exceptions Mycellius ---");
try {
    // On simule le fait de ne pas trouver une page
    throw new PageNotFoundException("PAGE-XXX");
} catch (PageNotFoundException e) {
    System.out.println("Erreur fonctionnelle : " + e.getMessage());
}
System.out.println("Le programme continue après la gestion de l'erreur.");
```

Tu auras PageNotFoundException en rouge car la Classe n'est pas importée. Passe ta souris sur l'erreur et clique sur "import class" et le rouge disparaîtra car la Class sera importée (vérifie en haut pour observer que l'import a bien été appliqué).

Explications :

- try { ... } : zone où une exception peut se produire.
- catch (PageNotFoundException e) : ce bloc s'exécute uniquement si ce type d'exception (ou un type compatible) est lancé.
- On montre que l'application ne « meurt » pas forcément quand une erreur arrive : on peut gérer et continuer.

Plus tard, les try/catch se trouveront plutôt dans la couche « contrôleur REST » ; la couche métier (service) se contentera de lancer les exceptions métier.

2.4 Décider de « politique d'erreur » pour Mycellius

Pour cette séance 3, on va adopter une politique simple :

- Si l'entrée est invalide (id vide, titre vide, etc.) → IllegalArgumentException (déjà en place dans Tag et WikiPage)
- Si la page n'existe pas alors qu'on la demande explicitement → PageNotFoundException
- Si on fait une recherche par titre, on ne jette pas d'exception, on renvoie une liste (éventuellement vide).

=> Ce sont des choix de contrat qui doivent être stables, car ils influenceront plus tard :

- les codes HTTP (400, 404...)
- les messages d'erreurs envoyés au front web ou mobile.

3. Collections et dépôt en mémoire : InMemoryWiki (≈ 45–60 min)

3.1 Pourquoi un « dépôt en mémoire » ?

En séance 2, tu manipules essentiellement des objets isolés (un Tag, une WikiPage).

Mais un wiki, c'est une collection de pages.

Tu n'as pas encore de base de données.

Donc pour l'instant, on va simuler le stockage par une collection en mémoire :

- Map<String, WikiPage> :
 - clé = id de la page (ex: "PAGE-001")
 - valeur = l'objet WikiPage

Avantages :

- Rapide à coder et à comprendre
- Parfait pour les tests JUnit
- Facile à remplacer plus tard par une vraie base ou un repository Spring Data (même interface, implémentation différente).

3.2 Créer la classe InMemoryWiki

Où ?

Dans `src/main/java/fr/mycellius/` on va créer un autre package `fr.mycellius.repository` pour bien séparer les rôles des packages).

Pourquoi séparer domain et repository ?

Aujourd'hui tu as déjà :

- `fr.mycellius.domain`
- `WikiPage`
- `Tag`
- `PageNotFoundException`
- etc.

Ces classes représentent le métier Mycellius :

- « ce qu'est une page »
- « ce qu'est un tag »
- « quelles règles métier s'appliquent »

=> `InMemoryWiki`, lui, ne décrit pas le métier.

Son rôle, c'est :

- stocker les `WikiPage` quelque part (ici, en mémoire dans une `Map`)
- retrouver une page par id
- chercher les pages dont le titre contient un mot
- etc.

=> C'est donc un « dépôt de données » = un repository.

Séparer les deux, ça permet de dire :

- `domain` : le langage métier (règles, objets métier)
- `repository` : la façon dont on stocke / retrouve ces objets (en mémoire, plus tard en base, puis en REST, etc.)

Le jour où tu remplaces `InMemoryWiki` par une vraie base de données, tu pourras changer juste le package `repository`, sans toucher aux classes métier dans `domain`.

Rappel : c'est quoi un package, concrètement ?

En Java, un package c'est deux choses :

- Un « dossier logique » annoncé en haut du fichier.

Exemple dans WikiPage : `package fr.mycellius.domain;`

- Un chemin de dossiers sur le disque :

`src/main/java/fr/mycellius/domain/WikiPage.java`

Donc :

- `package fr.mycellius.domain;` ↔ dossier `.../src/main/java/fr/mycellius/domain`
- `package fr.mycellius.repository;` ↔ dossier `.../src/main/java/fr/mycellius/repository`

On va donc créer un nouveau package `fr.mycellius.repository` qui correspondra au dossier :
`api/src/main/java/fr/mycellius/repository`

> Puis, crée un nouveau fichier et ajoute le code suivant : `InMemoryWiki.java`

```
package fr.mycellius.repository;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.domain.exception.PageNotFoundException;
import java.util.*;

public class InMemoryWiki {

    private final Map<String, WikiPage> pages = new HashMap<>();

    public WikiPage save(WikiPage page) {
        if (page == null) {
            throw new IllegalArgumentException("page null interdite");
        }
        pages.put(page.getId(), page);
        return page;
    }

    public boolean existsById(String id) {
        return pages.containsKey(id);
    }

    public WikiPage getById(String id) {
        WikiPage page = pages.get(id);
        if (page == null) {
            throw new PageNotFoundException(id);
        }
        return page;
    }

    public List<WikiPage> searchByTitleContaining(String fragment) {
        if (fragment == null) {
            throw new IllegalArgumentException("fragment de recherche null");
        }
        String needle = fragment.trim().toLowerCase(Locale.ROOT);
        List<WikiPage> result = new ArrayList<>();
        for (WikiPage page : pages.values()) {
            String title = page.getTitle();
            if (title != null && title.toLowerCase(Locale.ROOT).contains(needle)) {
                result.add(page);
            }
        }
        return result;
    }

    public List<WikiPage> findAll() {
        return new ArrayList<>(pages.values());
    }
}
```

Explications:

- `package fr.mycellius.repository`: la classe fait partie du package repository, qui regroupe les mécanismes de stockage/recherche.

- `private final Map<String, WikiPage> pages = new HashMap<>();`

- Map = sorte de tableau associatif avec association clé → valeur.
- String = ici la clé est un String (id de la page) et la valeur est l'Objet WikiPage.
- WikiPage = objet métier.
- HashMap = implémentation simple, non triée.
- final = on ne remplace pas la Map par une autre instance, mais on peut ajouter/supprimer des entrées.
- new HashMap<>() : implémentation concrète de Map, en mémoire.

- `public WikiPage save(WikiPage page) { ... }`

- Méthode pour sauvegarder (créer ou mettre à jour) une page.
- Si page est null → IllegalArgumentException.
- `pages.put(page.getId(), page);` : si l'id existe déjà, la valeur est écrasée (on accepte la mise à jour).
- On renvoie la page, ce qui rend le code plus fluide (chaînage possible).

- `public boolean existsById(String id) { ... }`

- Méthode utilitaire, servira dans le service pour refuser les doublons, par exemple.
- ```
public WikiPage getById(String id) { ... }
```
- `pages.get(id)` renvoie soit la page, soit null si elle n'existe pas.
  - Si null → on lance une `PageNotFoundException`.
  - Sinon → on renvoie la page.

- `public List<WikiPage> searchByTitleContaining(String fragment) { ... }`

- On refuse fragment null.
- On normalise la recherche : trim + toLowerCase.
- On parcourt toutes les pages : `pages.values()`.
- Si le titre en minuscules contient le fragment en minuscules → on ajoute à la liste de résultats.
- À la fin, on renvoie la liste (qui peut être vide, et c'est totalement acceptable).

- `public List<WikiPage> findAll() { ... }`

- On renvoie une nouvelle ArrayList pour éviter que l'appelant modifie directement notre Map interne.

En gros, les méthodes (save, existsById, getById, searchByTitleContaining, findAll) sont des « opérations de dépôt » :

- save : enregistre une page
- existsById : renvoie true/false si l'id est présent
- getById : renvoie la page ou lève PageNotFoundException
- searchByTitleContaining : recherche par fragment de titre
- findAll : renvoie une copie de la liste de toutes les pages

### 3.3 Mini test manuel dans App (avant JUnit)

Pour montrer le comportement à chaud, tu peux ajouter dans App.main quelque chose comme :

```
System.out.println("--- Démo InMemoryWiki ---");

InMemoryWiki repo = new InMemoryWiki();

WikiPage p1 = new WikiPage("PAGE-001", "Installation Docker", "...");
WikiPage p2 = new WikiPage("PAGE-002", "Installation SSH", "...");
repo.save(p1);
repo.save(p2);

System.out.println("Nombre de pages : " + repo.findAll().size());

System.out.println("Recherche 'docker' :");

for (WikiPage page : repo.searchByTitleContaining("docker")) {
 System.out.println(" - " + page.getId() + " : " + page.getTitle());
}
```

#### Pourquoi ?

- Tu visualises l'avantage de Map + List.
- Tu montres le lien avec le métier : recherche par titre.

## 4. Mini service métier : WikiService et contrat de méthodes (≈ 45–60 min)

### 4.1 Pourquoi un service ?

Dans une architecture propre (et encore plus avec Spring), on distingue :

- les entités / objets métier (Tag, WikiPage, etc.)
- les repositories (InMemoryWiki, plus tard une base de données)
- les services métier (WikiService), qui représentent des cas d'usage complets :
  - créer une page
  - rechercher des pages
  - mettre à jour le contenu, etc.

Le service va :

- appliquer des règles métiers supplémentaires (pas de doublon d'id, par exemple)
- utiliser InMemoryWiki pour stocker / lire les données
- définir clairement le contrat : quelles méthodes, quels paramètres, quels retours, quelles exceptions.

### 4.2 Créer WikiService

Où ?

On va créer un sous-package **fr.mycellius.service** (plus propre).

Puis crée la Classe : **WikiService.java**

```
package fr.mycellius.service;

import fr.mycellius.repository.InMemoryWiki;
import fr.mycellius.domain.exception.PageNotFoundException;
import fr.mycellius.domain.WikiPage;
import java.util.List;

public class WikiService {
 private final InMemoryWiki repository;

 public WikiService(InMemoryWiki repository) {
 if (repository == null) {
 throw new IllegalArgumentException("repository obligatoire");
 }
 this.repository = repository;
 }
}
```

```

/**
 * Crée une nouvelle page wiki.
 *
 * Règles :
 * - id obligatoire, non vide (WikiPage se charge déjà de vérifier ça)
 * - title obligatoire, propre (géré par WikiPage)
 * - si une page existe déjà avec le même id -> IllegalArgumentException
 *
 * @param id identifiant unique de la page
 * @param title titre de la page
 * @param content contenu initial (peut être vide mais pas null)
 * @return la page créée
 */
public WikiPage createPage(String id, String title, String content) {
 if (repository.existsById(id)) {
 throw new IllegalArgumentException("Une page existe déjà avec l'id " + id);
 }
 WikiPage page = new WikiPage(id, title, content);
 return repository.save(page);
}

/**
 * Retourne la page correspondant à l'id fourni.
 *
 * @param id identifiant de la page
 * @return la page trouvée
 * @throws PageNotFoundException si aucune page avec cet id
 */
public WikiPage getPageById(String id) {
 return repository.getById(id);
}

/**
 * Recherche des pages dont le titre contient le fragment fourni (insensible à la casse).
 *
 * @param fragment morceau de texte à chercher dans le titre
 * @return liste (éventuellement vide) de pages correspondantes
 */
public List<WikiPage> searchByTitle(String fragment) {
 return repository.searchByTitleContaining(fragment);
}

/**
 * Retourne toutes les pages existantes.
 */
public List<WikiPage> listAllPages() {
 return repository.findAll();
}
}

```

### Explications :

- Le constructeur impose repository non null → plus de risques de NullPointerException cachés.
- createPage applique une règle supplémentaire : pas de doublon d'id.
  - On réutilise WikiPage pour vérifier id/titre/content.
  - Si id déjà utilisé → IllegalArgumentException explicite.
- getPageById délègue à repository.getByid, donc propage PageNotFoundException.
- searchByTitle et listAllPages ne jettent pas d'exceptions si rien n'est trouvé : elles renvoient une liste vide.

## 4.3 Utiliser WikiService dans App.main

### Objectif

- Remplacer les petites démos dispersées par une mini démonstration cohérente du service.
- Dans App.main, après tes affichages actuels, tu peux faire :

```
System.out.println("--- Démo WikiService ---");
InMemoryWiki repository = new InMemoryWiki();
WikiService service = new WikiService(repository);
service.createPage("PAGE-001", "Installation Docker", "Étapes pour installer Docker sur Ubuntu.");
service.createPage("PAGE-002", "Installation SSH", "Configurer l'accès SSH sécurisé.");
// Cas nominal : recherche
System.out.println("Pages contenant 'docker' :");
for (WikiPage page : service.searchByTitle("docker")) {
 System.out.println(" - " + page.getId() + " : " + page.getTitle());
}
// Cas d'erreur : doublon
try {
 service.createPage("PAGE-001", "Autre titre", "Contenu");
} catch (IllegalArgumentException e) {
 System.out.println("Erreur attendue (doublon id) : " + e.getMessage());
}
// Cas d'erreur : page introuvable
try {
 service.getPageById("PAGE-999");
} catch (PageNotFoundException e) {
 System.out.println("Erreur attendue (page introuvable) : " + e.getMessage());
}
```



Explications :

Tu montres en une fois :

- le métier nominal (création + recherche)
- les erreurs bien gérées (doublon, page introuvable)
- l'intérêt d'un service comme point d'entrée métier.

## 5. Tests JUnit sur WikiService (≈ 45–50 min)

### 5.1 Où placer les tests ?

Dans un projet Maven :

- code de production → src/main/java
- code de test → src/test/java

On va donc créer :

api/src/test/java/fr/mycellius/service/WikiServiceTest.java

Dans IntelliJ :

- clic droit sur src/test/java → New → Package → fr.mycellius.service
- clic droit sur fr.mycellius.service → New → Java Class → WikiServiceTest

IntelliJ génère :

```
package fr.mycellius.service;

public class WikiServiceTest {

}
```

### 5.2 Ajouter les imports JUnit

On garde la même logique que pour WikiPageTest et TagTest.

Intégrer le code suivant dans WikiServiceTest.java :

```
package fr.mycellius.service;

import fr.mycellius.repository.InMemoryWiki;
import fr.mycellius.domain.exception.PageNotFoundException;
import fr.mycellius.domain.WikiPage;
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

public class WikiServiceTest {

 @Test
 void createPage_nominal_cree_une_page() {
 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);
 }
}
```

```

 WikiPage page = service.createPage("PAGE-001", "Installation Docker", "contenu");

 assertEquals("PAGE-001", page.getId());
 assertEquals("Installation Docker", page.getTitle());
 assertEquals(1, repo.findAll().size());
 }

 @Test
 void createPage_refuse_doublon_id() {
 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);

 service.createPage("PAGE-001", "Titre 1", "contenu");

 assertThrows(IllegalArgumentException.class, () ->
 service.createPage("PAGE-001", "Titre 2", "autre contenu")
);
 }

 @Test
 void getPageById_page_existante_ok() {
 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);

 service.createPage("PAGE-001", "Titre", "contenu");

 WikiPage page = service.getPageById("PAGE-001");

 assertEquals("PAGE-001", page.getId());
 }

 @Test
 void getPageById_page_inexistante_declenche_exception() {
 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);

 assertThrows(PageNotFoundException.class, () ->
 service.getPageById("PAGE-999")
);
 }

 @Test
 void searchByTitle_retourne_liste_vide_si_rien() {

```

```

 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);

 List<WikiPage> result = service.searchByTitle("docker");

 assertTrue(result.isEmpty());
 }

 @Test
 void searchByTitle_trouve_pages_correspondantes() {
 InMemoryWiki repo = new InMemoryWiki();
 WikiService service = new WikiService(repo);

 service.createPage("PAGE-001", "Installation Docker", "...");
 service.createPage("PAGE-002", "Docker avancé", "...");

 List<WikiPage> result = service.searchByTitle("docker");

 assertEquals(2, result.size());
 }
}

```

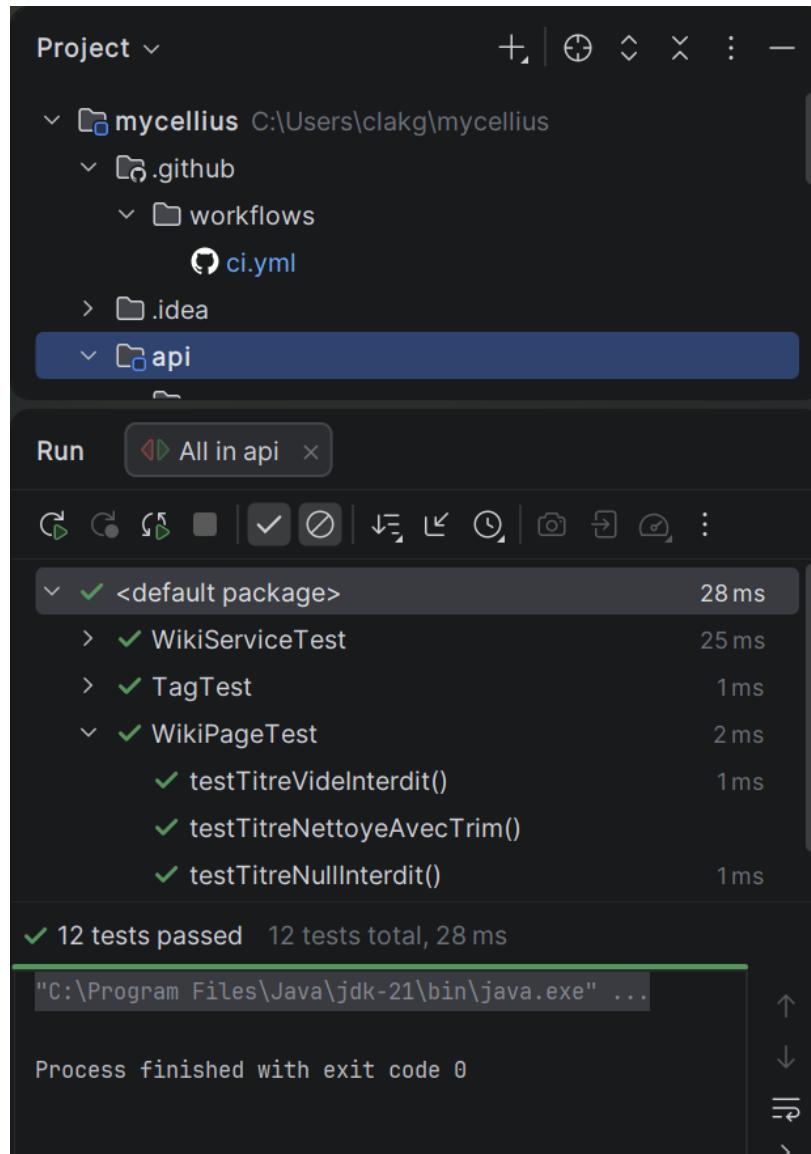
### Explications:

- Chaque test suit un scénario métier simple.
- On réutilise InMemoryWiki dans les tests : pas besoin de base, pas de Spring, tout est ultra rapide.
- On couvre :
  - createPage nominal
  - createPage en erreur (doublon)
  - getPageById nominal et en erreur
  - searchByTitle avec résultat vide et avec résultats
- On s'appuie beaucoup sur assertThrows pour les cas d'erreurs, ce qui renforce l'idée de contrat :
  - « dans ce cas, ça doit jeter telle exception ».

### 5.3 Lancer les tests

Localement, tu peux :

- soit lancer tous les tests depuis IntelliJ (clic droit sur le module api → Run "All Tests")



- soit utiliser Maven dans le terminal :

```
mvn -f api/pom.xml test
```

Tu dois obtenir un build SUCCESS avec tous les tests au vert.

```
PS C:\Users\clakg\mycellius> mvn -f api/pom.xml test
```

```
[INFO] Scanning for projects...
```

```
[INFO]
```

```

[INFO] -----< fr.mycellius:api >-----
[INFO] Building api 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[jar]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ api ---
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ api ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 8 source files with javac [debug target 21] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ api ---
[INFO] skip non existing resourceDirectory C:\Users\clakg\mycellius\api\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ api ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 3 source files with javac [debug target 21] to target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ api ---
[INFO] Using auto detected provider
org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running fr.mycellius.domain.TagTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s -- in
fr.mycellius.domain.TagTest
[INFO] Running fr.mycellius.domain.WikiPageTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in
fr.mycellius.domain.WikiPageTest
[INFO] Running fr.mycellius.service.WikiServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s -- in
fr.mycellius.service.WikiServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]

```

[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 2.890 s  
[INFO] Finished at: 2026-01-03T01:53:27+01:00  
[INFO] -----

## 6. CI + workflow Git (dev → test) avec ce nouveau code (≈ 20–30 min)

### Objectif du chapitre 6

- travailler sur la branche dev ;
- lancer les tests Maven en local ;
- committer et pousser son code vers GitHub ;
- déclencher et vérifier la CI GitHub Actions ;
- créer une Pull Request dev → test et la merger uniquement si la CI est verte ;
- (optionnel) créer un tag v0.0.x pour marquer une mini-version.

### Rappel – contexte de la séance 3

Tu as maintenant dans api/ :

- des classes métier : Tag, WikiPage, PageNotFoundException, InMemoryWiki, WikiService...
- des tests JUnit (par exemple WikiServiceTest).
- La CI sur GitHub est déjà configurée pour lancer Maven sur l'API.

### 6.1 Rappel – CI GitHub Actions

Dans le dépôt, le fichier `.github/workflows/ci.yml` contient déjà un job qui :  
se déclenche sur les branches dev, test, prod (push et/ou pull\_request) ;  
exécute Maven sur le module api, par exemple avec une ligne de ce type :

```
mvn -f api/pom.xml test
```

### Rappel

Tu n'as rien à modifier dans ce workflow pour la séance 3.

Dès que tu feras un git push sur dev, GitHub :

- téléchargera ton dépôt,
- lancera `mvn -f api/pom.xml test`,
- indiquera un statut "vert" (succès) ou "rouge" (échec) dans :
  - l'onglet Actions,
  - et sur les Pull Requests (section Checks / GitHub Actions).

### Rappel – mvn -f api/pom.xml test

`mvn` : c'est la commande Maven ;

`-f api/pom.xml` : Maven doit utiliser le `pom.xml` qui se trouve dans le dossier `api` ;

`test` : Maven exécute la phase "test" (compilation + tests unitaires).



## 6.2 Cycle de travail local en ligne de commande (sur dev)

Tout ce qui suit se fait :

- dans un terminal (PowerShell, Git Bash, ou terminal VS Code),
- dans le dossier racine du projet, c'est-à-dire C:\Users\ton-utilisateur\mycellius.

### Étape 1 – Vérifier que tu es dans le bon dossier

Dans le terminal :

```
cd C:\Users\ton-utilisateur\mycellius
```

Remplace ton-utilisateur par ton nom de session Windows (chez toi, c'est clag).

### Étape 2 – Se placer sur la branche dev

Commande :

```
git checkout dev
```

Ce que ça fait :

- bascule la branche de travail sur dev ;
- met à jour les fichiers du dossier pour correspondre à cette branche.

Rappel

C'est très important : si tu restes sur test ou prod et que tu codes, tu "pollues" ces branches avec du travail en cours.

### Étape 3 – Récupérer les dernières modifications de dev (pull)

Commande :

```
git pull origin dev
```

Ce que ça fait :

- récupère du serveur GitHub tous les nouveaux commits de dev ;
- essaie de les fusionner avec ta dev locale.

Rappel

On fait ce "pull" avant de commencer la séance, surtout si on a travaillé :  
sur un autre poste,  
ou si quelqu'un d'autre a poussé du code sur dev.

#### Étape 4 – Coder dans api/ (avec IntelliJ)

Cette étape se fait dans IntelliJ uniquement pour l'édition de code :

tu modifies ou crées des fichiers dans :

api/src/main/java/fr/mycellius/... (WikiPage, Tag, InMemoryWiki, WikiService, etc.)

api/src/test/java/fr/mycellius/... (les tests JUnit, ex. WikiServiceTest)

#### Important

Même si tu écris ton code dans IntelliJ, c'est toujours le même dossier mycellius/ que Git voit dans le terminal.

Git se fiche de l'IDE, il regarde juste les fichiers sur le disque.

#### Étape 5 – Lancer les tests Maven en local

Avant de committer, on vérifie que tout compile et que tous les tests passent.

Commande (dans le dossier mycellius) :

```
mvn -f api/pom.xml test
```

Ce que ça fait :

- Maven lit api/pom.xml ;
- compile les classes Java de api/src/main/java ;
- compile les tests dans api/src/test/java ;
- exécute les tests JUnit.

Résultat attendu :

si tout va bien, à la fin, tu dois voir :

```
BUILD SUCCESS
```

si quelque chose ne va pas (erreur de compilation, test KO), tu verras :

```
BUILD FAILURE
```

avec des messages d'erreur (en rouge) expliquant ce qui ne va pas.

### Rappel

On insiste en séance 3 sur la robustesse.

### **Règle d'or :**

si `mvn -f api/pom.xml test` est en échec localement, on NE fait PAS de commit.

on corrige le code, puis on relance la commande jusqu'à avoir un BUILD SUCCESS.

### **Étape 6 – Vérifier les fichiers modifiés (git status)**

Une fois les tests OK, on regarde ce que Git voit comme modifications.

### Commande :

```
git status
```

### Ce que ça montre :

- la branche courante (dev) ;
- les fichiers modifiés mais non ajoutés ("modified") ;
- les fichiers nouvellement créés ("untracked files").

### Rappel

Cette commande est l'équivalent d'un "tableau de bord" :

- tu vérifies que tu n'es pas en train d'embarquer des fichiers que tu ne voulais pas (ex. un fichier temporaire, un .log, etc.) ;
- tu vois précisément quels fichiers font partie du travail de la séance.

### **Étape 7 – Préparer les fichiers pour le commit (git add)**

On veut ajouter toutes les modifications Java de l'API à ce commit.

Comme elles sont toutes sous `api/`, on peut faire :

```
git add api/
```

### Ce que ça fait :

toutes les modifications situées dans le dossier `api/` (fichiers main + tests) passent dans la "staging area".

### Rappel

`git add` ne "termine" pas le travail.

Il marque juste les fichiers comme "prêts à être commités" dans le prochain git commit.

## Étape 8 – Vérifier une dernière fois avant commit

### Commande :

```
git status
```

### Attendu :

- les fichiers de api/ doivent maintenant être dans la section “Changes to be committed” ;
- il ne doit pas rester de fichier Java oublié dans “untracked” ou “modified” (sauf si tu veux les garder pour plus tard).

## Étape 9 – Créer le commit

### Commande :

```
git commit -m "feat(java): add InMemoryWiki and WikiService"
```

### Explication :

git commit : crée un nouveau “snapshot” (point d’historique) avec les fichiers ajoutés ;

-m "...": message de commit :

feat(java): ... → indique qu’on ajoute une nouvelle fonctionnalité côté Java ;

le reste décrit ce qu’on a fait (ajout InMemoryWiki, WikiService, tests...).

### Rappel

Un bon message de commit aide :

- toi, plus tard, quand tu reliras l’historique ;
- un autre développeur ;
- l’enseignant ou jury qui voudra comprendre l’évolution du projet.

## Étape 10 – Pousser le commit sur GitHub (git push)

### Commandes (toujours sur dev) :

```
git push origin dev
```

### Ce que ça fait :

- prend tes commits locaux sur la branche dev ;
- les envoie sur la branche dev du dépôt GitHub (remote origin).

### À partir de là :

GitHub voit ton nouveau code ;

GitHub Actions détecte le push sur dev et lance automatiquement la CI (mvn -f api/pom.xml test).

#### Rappel

La différence :

mvn -f api/pom.xml test en local : tu vérifies toi-même avant d'envoyer ;

CI GitHub : un robot impartial revérifie sur un environnement "propre" (très utile pour détecter un "chez moi ça marche" qui n'est pas vrai ailleurs).

### **6.3 Vérifier la CI et créer la Pull Request dev → test**

Cette partie se fait dans le navigateur, sur GitHub (comme en séance 1 et 2).

#### **Étape 11 – Vérifier la CI dans l'onglet Actions**

- Va sur ton dépôt GitHub (URL habituelle).

- Clique sur l'onglet Actions:

Tu dois voir une nouvelle exécution "CI" déclenchée par :

- un push sur dev, ou
- une Pull Request dev → test (plus tard).

#### Attendu :

si tout se passe bien : un check vert (✅) sur le run CI associé à la branche dev ;

si c'est rouge (❌) :

- tu ouvres le run,

tu regardes les logs pour voir :

- erreur de compilation,
- test JUnit qui a échoué,
- problème de chemin Maven, etc.

#### Rappel

Si la CI est rouge sur dev, ça signifie :

"sur un environnement propre, ton projet ne compile pas ou les tests ne passent pas".

On retourne alors en local, on corrige, et on refait :

```
mvn -f api/pom.xml test,
git add api/,
git commit / git push
```

## Étape 12 – Créer une Pull Request dev → test

- Dans le dépôt GitHub, clique sur l'onglet Pull requests.
- Clique sur le bouton New pull request.
- Dans la page de création de PR :
  - base : choisis test (branche de destination) ;
  - compare : choisis dev (branche source avec ton nouveau code).
- GitHub affiche :
  - les commits concernés ;
  - les fichiers modifiés.
- Ensuite :
  - clique sur Create pull request,
  - ajoute éventuellement un titre (ex. "S3 – InMemoryWiki + WikiService + tests"),
  - valide.

### Rappel

La PR est un "ticket" où tu dis :

"Je propose de fusionner le travail de dev dans test."

## Étape 13 – Vérifier les checks CI sur la PR

Une fois la PR créée :

- GitHub relance la CI (si configurée sur pull\_request vers test) ;
- tu vois une section "Checks" ou "GitHub Actions" dans la PR.

États possibles :

- rond jaune : CI en cours ;
- check vert : CI OK ;
- croix rouge : CI KO (mêmes causes possibles que sur dev).

### Règle d'or

On ne merge JAMAIS une PR si la CI est rouge.

On doit :

- retourner en local ;
- corriger le code ;

- recommencer le cycle :
- tests Maven locaux,
- commit,
- push sur dev,
- CI,

PR mise à jour automatiquement par les nouveaux commits.

#### **Étape 14 – Merger la PR vers test**

Quand tous les checks sont verts :

- dans la PR, clique sur Merge pull request ;
- confirme.

Résultat :

la branche test contient maintenant :

- les classes et tests de la séance 3,
- validés par la CI.

C'est cette branche test qui est censée être "stable pour recette interne".

### **6.4 Optionnel – Créer un tag version v0.0.x**

#### **Objectif**

Marquer un point important de l'évolution du projet, par exemple après stabilisation de la séance 3 sur test.

#### **Étape 15 – Se placer sur test en local et récupérer la dernière version**

Dans le terminal (toujours dans C:\Users\ton-utilisateur\mycellius) :

```
git checkout test
git pull origin test
```

Rappel

On se place d'abord sur test, on s'assure que test local = test sur GitHub.

#### **Étape 16 – Créer un tag local**

Commande (exemple) :

```
git tag v0.0.1
```

Ce que ça fait :

crée une étiquette v0.0.1 sur le dernier commit de la branche test.

Rappel

Le nom du tag est à toi : v0.0.1, v0.1.0, S3-demo, etc.

mais il vaut mieux rester cohérent :

- v0.0.1 pour la première mini-version validée,
- v0.0.2 pour la suivante, etc.

**Étape 17 – Pousser le tag sur GitHub**

Commande :

```
git push origin v0.0.1
```

Résultat :

- sur GitHub, tu verras ce tag dans la liste des tags ;
- on peut imaginer : “v0.0.1 = Mycellius après séance 3 (service en mémoire + tests de base OK)”.

**Conclusion de la partie 6**

En ligne de commande, tu as maintenant un rituel complet :

```
git checkout dev
```

```
git pull origin dev
```

```
coder dans api/ avec IntelliJ
```

```
mvn -f api/pom.xml test
```

```
git status
```

```
git add api/
```

```
git commit -m "..."
```

```
git push origin dev
```

```
vérifier la CI (Actions)
```

```
créer PR dev → test
```

```
vérifier que la CI est verte
```

```
merger la PR
```

```
(optionnel) tagger v0.0.x après merge sur test.
```



Ce rituel sera exactement le même en séance 4 quand on passera à Spring Boot : on remplacera juste le contenu Java du module api, pas le workflow Git/CI.

## **7. Budget « refactor qualité » (à utiliser si temps restant)**

Quelques pistes de petites améliorations, en cohérence avec ce qui viendra en Spring :

- vérifier que les packages sont propres :
  - fr.mycellius.domain (Tag, WikiPage, InMemoryWiki, PageNotFoundException)
  - fr.mycellius.service (WikiService)
- renommer des méthodes pour plus de clarté si besoin (par exemple searchByTitle → searchPagesByTitle)
- supprimer le code de démo temporaire dans App.main une fois que tu as des tests stables
- ajouter des commentaires Javadoc sur les méthodes publiques importantes (createPage, getPageById, etc.)
- vérifier que tous les fichiers sont bien formatés (indentation, noms de variables lisibles).
- Ce travail de « petite qualité » est extrêmement précieux :
  - il rend le passage à Spring Boot plus fluide (tu réutiliseras directement WikiService)
  - il donne aux étudiants l'habitude d'avoir un code propre avant de passer à la suite.

## **Conclusion de la séance 3**

À ce stade, tu as :

- un mini noyau métier robuste (exceptions, collections, service)
- un début de « contrat d'API » côté Java (méthodes + exceptions associées)
- une couverture de tests raisonnable sur ce noyau
- un workflow Git + CI fonctionnel autour de ce code

## **La séance 4 pourra alors se concentrer sur :**

- transformer ce service en vraie API REST avec Spring Boot
- exposer createPage, getPageById, searchByTitle via des endpoints HTTP
- commencer à gérer les statuts HTTP (200, 201, 400, 404...) à partir des exceptions définies ici.